

Adapting to Electricity Market Regime Shifts: Model-Based Reinforcement Learning for Battery Storage Arbitrage with a Validated Learned Market Model

Nicholas Lai Jin Yung
MSc Artificial Intelligence
Queen Mary University of London

July 27, 2026

Abstract

Grid-scale battery energy storage earns revenue through electricity-price arbitrage, but any controller tuned on historical prices must eventually face a market regime it was not trained on, as European operators discovered abruptly in 2022, when gas prices moved far outside every historical distribution. This dissertation asks how a reinforcement-learning storage controller should be built so that it can adapt *rapidly* to such persistent regime shifts, and answers it by exploiting a structural property of the problem: a battery small relative to the grid is a *price-taker*, so its environment decomposes into battery dynamics that are known exactly and an exogenous market price process that must be learned, and, crucially, that is observed for free, one day per day, regardless of what the policy does. The world model is therefore not a joint state-action transition model but a generative model of daily 24-hour price vectors: an ensemble of hour-autoregressive recurrent networks, admitted into policy training only after passing a statistical validation gate against held-out market data (flat diagonal-Gaussian architectures fail this gate, a documented negative result). A Soft Actor-Critic agent trains Dyna-style inside this learned market, with its replay buffer seeded by dynamic-programming planner demonstrations on the stored real days, so the physical asset never sees an untrained exploratory action. Three experiments build the argument. First, a sample-efficiency study under an explicit real-data budget shows the learned-model agent substantially outperforms both online training and a replay-only control at equal data, and identifies the budget below which the market model fails validation. Second, a zero-shot robustness study under unseen transient shocks (cold snaps, fuel spikes, plant outages, drought, heat waves) shows the model-trained policy degrades more gracefully than its baselines: a floor, not a solution. Third, the central experiment: a streaming deployment in which the market shifts persistently mid-run, and an adaptive agent refits the market model nightly on the freely-observed price stream and fine-tunes its policy in imagination overnight, logging the ensemble’s per-day predictive likelihood as the shift-detection signal: recovering in days what a model-free agent, bound by 24 real transitions per day, recovers in weeks or not at all. Throughout, performance is normalized by exact optimization oracles (a hindsight optimum and a rolling-horizon dynamic program over the agent’s own information set), so results are reported as fractions of attainable value rather than raw dollars. All experiments run in a calibrated, fully deterministic merit-order market simulator whose regime-shift scenarios consume no randomness, enabling exact paired counterfactual evaluation.

Contents

1	Introduction	4
1.1	Background and Motivation	4
1.2	The Problem	4
1.3	Proposed Solution	5
1.4	Dissertation Hypothesis and Research Contributions	6
2	Related Work	7
2.1	Reinforcement Learning for Battery Arbitrage	7
2.2	Model-Based Reinforcement Learning and World Models	7
2.3	Non-Stationarity and Adaptation in Reinforcement Learning	8
3	Synthetic Merit-Order Market Simulator	8
3.1	Justification for Synthetic Data	8
3.2	Exogenous Drivers	8
3.3	Demand	9
3.4	The Generator Fleet	9
3.5	Market Clearing	9
3.6	Calibration	9
3.7	Regime-Shift Scenarios	9
4	Problem Formulation: The Arbitrage MDP	10
4.1	Battery Physics	10
4.2	State Space	10
4.3	Action Space	10
4.4	Reward	10
4.5	Episodes and Units	11
5	Methodology	11
5.1	Oracles and Baselines	11

5.2	The Learned Market Model	12
5.3	The Validation Gate	12
5.4	Dyna Training with Planner Demonstrations	13
5.5	The Adaptation Loop	13
6	Experimental Setup	14
6.1	Experiment 1: Sample Efficiency under a Real-Data Budget	14
6.2	Experiment 2: Zero-Shot Robustness to Transient Shocks	14
6.3	Experiment 3: Adaptation to Persistent Regime Shifts	14
6.4	Hyperparameters, Seeds, and Reporting	15
7	Results and Discussion	16
7.1	Oracle Ladder and Attainable Value	16
7.2	The Validation Gate and the Data Threshold	16
7.3	Sample Efficiency under the Budget	16
7.4	Zero-Shot Robustness	17
7.5	Adaptation to Persistent Shifts (Central Result)	17
7.6	Physics-Informed Battery Dynamics	18
7.7	Discussion	23
8	Conclusion and Future Work	23
8.1	Summary of Findings	23
8.2	Limitations	24
8.3	Future Directions	24
	Acknowledgements	25
A	Additional Material	26
A.1	Nightly Ensemble-Refit Calibration Drift	26

1 Introduction

This chapter introduces the motivation for grid-scale battery storage, the technical and economic challenges of electricity arbitrage under a market that experiences unseen regime shifts, and the central claim of the dissertation.

1.1 Background and Motivation

The global transition away from fossil fuels and towards sustainable energy sources is reshaping how modern power grids are planned and operated. Renewable technologies, particularly solar and wind, are essential for decarbonisation, but they also introduce a major operational challenge because their output is intermittent (IEA, 2023). In practice, renewable generation does not always align with demand. Solar and wind output can be high when demand is relatively low, while the grid may experience stress during evening peaks or periods of low wind availability.

Grid-scale Battery Energy Storage Systems (BESS) provide one way to address this mismatch between generation and demand. By charging during periods of surplus generation and discharging when demand is high, batteries act as an essential energy buffer helping stabilise the grid against volatility and supporting higher levels of renewable penetration (Aneke and Wang, 2016). This makes BESS an increasingly important part of future electricity systems.

However, operating BESS effectively is not straightforward. The economic viability of a battery often depends on electricity price arbitrage, where the system charges when prices are low and discharges when prices are high. A policy that focuses only on short-term profit may cycle the battery aggressively, which can accelerate capacity fade and reduce its State of Health (SoH) (Xu et al., 2018). The long-term value of energy storage depends on control strategies that account for ageing mechanisms alongside revenue.

Compounding this challenge, the battery operates in a shifting market where prices are influenced by a variety of factors creating regimes that may not have been encountered during the training of a control policy. This raises a question that goes beyond whether reinforcement learning can learn a good arbitrage policy: can the learned controller remain effective when the market shifts to a regime it has never seen?

1.2 The Problem

Reinforcement learning (RL) is a suitable framework for battery arbitrage because the problem is sequential by nature. At each timestep, the controller must decide whether to charge, discharge, or remain idle, while accounting for uncertain electricity prices and the long-term impact of degradation. This fits naturally within the Markov Decision Process (MDP) formalism. Recent work has shown that deep RL agents can learn profitable energy-storage arbitrage strategies and can outperform hand-crafted heuristics (Cao et al., 2020), while Soft Actor-Critic (SAC) is a common choice for continuous-control problems because of its off-policy learning and entropy regularisation (Haarnoja et al., 2018).

However, applying RL to battery arbitrage is complicated by an environment that shifts under the agent. This domain presents three core challenges.

Sample inefficiency. Model-free algorithms such as SAC learn mainly through environment

interaction. In this project, one training episode is 14 days of hourly dispatch decisions (336 steps), and training SAC from scratch to convergence takes hundreds of thousands of such steps (Section 6.4) on the order of years of once-a-day real market data if learned model-free and from scratch, which is the inefficiency this dissertation sets out to remove.

Constraint violations during exploration. Exploration, particularly early in training, can produce physically infeasible actions. Examples include discharge requests that would push the State of Charge (SoC) below its lower operating limit or charge requests that would exceed the upper limit.

Market regime shifts. Battery arbitrage depends on a market process that is itself non-stationary. Electricity prices reflect seasonal demand patterns, renewable availability, weather conditions, and fuel costs, all of which can drift gradually or shift abruptly. A policy calibrated to one regime can encounter very different price spreads and cycling incentives after such a shift.

Together, these challenges motivate a control approach that can learn from few real market days, never explores untrained on the physical system, and adapts to persistent regime shifts at a speed set by days of observation rather than weeks of on-asset fine-tuning.

1.3 Proposed Solution

The objective is fixed from the beginning as maximizing arbitrage profit and minimizing battery degradation, with wear priced at the cost of the health it consumes. What this dissertation develops is a way to keep pursuing that objective when the market changes under the agent. The proposed controller is a Dyna-style Soft Actor-Critic agent that trains inside a learned world model of the market, and rapid adaptation follows from refitting the market ensemble. Each night the agent refits the ensemble on the newly observed prices and relearns its policy in imagination. The ensemble’s predictive likelihood on each incoming day is recorded as the shift-detection signal.

The design rests on one structural observation. A battery operating as a price-taker faces two distinct domains: a deterministic physical system and a stochastic market environment. We decompose the world model accordingly:

- **The asset physics are known and deterministic.** The State of Charge (SoC) and State of Health (SoH) evolve through well understood electrochemistry, including round-trip efficiency, throughput-based cycling degradation, and calendar ageing. Nothing here needs to be learned.
- **Only the market is stochastic and learned.** Rather than trying to model complex grid-level market drivers, the agent only requires a generative model of 24-hour price vectors. We implement this as an ensemble of hour-autoregressive recurrent networks conditioned on calendar features and historical prices.

This decomposition means only the market half ever needs to adapt, while the battery half remains fixed. It also sets the data economics of adaptation. A model-free agent gathers at most 24 real transitions per day, no matter how urgently its policy needs updating. The generative model turns those same 24 observed prices into unlimited imagined training days overnight, so the effective sample rate is bounded by compute rather than by the calendar.

1.4 Dissertation Hypothesis and Research Contributions

The dissertation investigates a core hypothesis:

For a price-taking storage asset, a Dyna-style RL agent whose world model is a generative model of daily market prices, with battery degradation modelled exactly, can attempt to maximise arbitrage profit and minimize degradation using far less real market data than model-free training, and can adapt to persistent market regime shifts within days by refitting the price model on newly observed days and updating the policy in imagination.

The dissertation is organised around four questions this hypothesis raises, each investigated empirically rather than asserted as an improvement delivered:

1. **Does a price-taker decomposition make model-based RL trustworthy enough to train a policy on?** We investigate a world model that separates exactly-known battery physics from a learned generative model of exogenous daily market prices. Because the price process cannot be steered by the policy’s actions, we ask whether this decomposition removes the model-exploitation failure mode that limits general model-based RL, and whether it lets the agent train on full 24-step imagined days rather than short branched rollouts. To ask whether the model can be trusted as a checkable question rather than a judgement call, we introduce a pre-training statistical validation gate that admits a fitted ensemble to policy training only if it reproduces the real market’s calibration statistics; a documented negative result shows flat diagonal-Gaussian architectures fail this gate by over-dispersing the daily spread, while an hour-autoregressive factorisation passes it (Sections 5.2–5.3).
2. **Under a fixed real-data budget, is it a validated world model — not more compute — that buys sample efficiency?** We investigate whether, under an identical real-data and gradient budget, Dyna training inside the gated market model outperforms both online model-free training and a replay-only control that receives the same data and unlimited gradient epochs. The gate and the replay control together isolate the source of any gain: the replay control rules out “more gradient updates” as the explanation, and the gate certifies that what the dyna arm has instead is a model whose days are statistically indistinguishable from real ones. The finding is affirmative — a validated generative model, not additional compute, is what a small real-data budget buys (Section 7.3).
3. **Can an in-imagination adaptation loop recover from persistent regime shifts?** We investigate a nightly refit-and-replan procedure that updates the market model and fine-tunes the policy entirely in imagination, evaluated in a streaming deployment against oracle-normalized capture, recovery-lag, and cumulative-regret metrics, on a simulator whose regime-shift scenarios consume no randomness and therefore support exact paired counterfactual evaluation. The finding is negative — the loop does not beat a frozen model on any shift — with the cause traced to nightly-refit calibration drift (Section 7.5).
4. **Do physics-informed battery dynamics improve arbitrage performance?** As the mirror image of the price-taker decomposition — holding the market exact and instead *learning* the battery from a log of transitions — we investigate whether modelling the battery dynamics as a physics-informed network, rather than a plain MLP surrogate, improves the sample efficiency of the learned dynamics and the resulting agent. The

finding is negative: on this smooth, low-dimensional map there is no robust predictive sample-efficiency crossover, and the physics prior’s advantage is confined to guaranteed physical admissibility rather than accuracy (Section 7.6).

2 Related Work

This chapter positions the dissertation within work on RL for battery arbitrage, model-based RL, and non-stationarity in reinforcement learning.

2.1 Reinforcement Learning for Battery Arbitrage

The application of RL to electricity market bidding and battery dispatch has attracted growing attention. Early work by [Cao et al. \(2020\)](#) demonstrated that deep Q-networks could learn profitable day-ahead bidding strategies, outperforming linear-programming baselines under price uncertainty. Subsequent studies have applied policy-gradient methods, such as Proximal Policy Optimization (PPO), and off-policy actor-critic architectures, such as Soft Actor-Critic (SAC), for real-time arbitrage. Among these, SAC has emerged as a strong default choice for continuous-action energy control problems due to its entropy regularisation and off-policy sample reuse ([Haarnoja et al., 2018](#)).

A central focus within this literature is the tension between short-term profit maximisation and long-term asset preservation. While model-free controllers can be trained on reward formulations that charge for physical battery degradation, they remain vulnerable to non-stationary environments. In modern energy markets, sudden shifts in supply and demand can strand a policy tuned to the old regime. Under standard model-free paradigms, adapting to a new regime requires extensive online retraining which is rate-limited by the market itself, yielding only 24 real hourly transitions per day, and during which the agent may execute suboptimal or physically damaging cycles on the asset. This dissertation studies robustness and adaptation to such shifts as its central question, rather than as an afterthought to a stationary benchmark.

2.2 Model-Based Reinforcement Learning and World Models

Model-based RL (MBRL) improves sample efficiency by learning a dynamics model $\hat{f}(s, a) \rightarrow s'$ and using it to generate additional training data. The Dyna architecture ([Sutton, 1991](#)) is the canonical framework: the agent updates a dynamics model from real experience and generates synthetic transitions from it to augment the replay buffer. This approach has been shown to reduce the required number of real interactions by an order of magnitude in continuous-control benchmarks ([Janner et al., 2019](#); [Chua et al., 2018](#)).

In the general setting, the quality of synthetic data is limited by model errors that *compound through the agent’s actions* over multi-step rollouts motivating short branched rollouts from probabilistic ensembles ([Janner et al., 2019](#)) and uncertainty-aware planning ([Chua et al., 2018](#)). A key point of this dissertation is that the price-taker decomposition removes this failure mode: the learned process (daily prices) is exogenous, so the policy’s actions cannot steer the model into self-confirming error.

World-model approaches such as Dreamer ([Hafner et al., 2020](#)) learn latent dynamics and plan entirely in imagination. While powerful, these methods introduce substantial architectural

complexity and are harder to interpret in safety-critical energy applications. The approach taken here retains a directly inspectable decomposition: exact battery physics plus a small generative model of prices whose fidelity is measured against held-out data before it is ever trusted.

2.3 Non-Stationarity and Adaptation in Reinforcement Learning

Reinforcement learning under non-stationary dynamics has been studied as worst-case planning against continuous drift (Lecarpentier and Rachelson, 2019), as a continual-learning problem balancing plasticity against forgetting (Khetarpal et al., 2022), and surveyed more broadly by Padakandla (2021); the supervised-learning literature on concept drift (Gama et al., 2014) contributes the detect-then-adapt pattern this work follows. The setting here is distinctive in two respects. First, the non-stationary component is *exogenous and fully observed*: every day the market publishes the very quantity the model must track, so adaptation data accrues at a fixed rate for free, independent of the policy. Second, the asset constraint forbids on-policy re-exploration, so adaptation must run off-asset as follows: refit the small market model, retrain in imagination, deploy tomorrow. The dissertation’s adaptation experiment measures how much this structure is worth relative to model-free online fine-tuning bound to the real stream.

3 Synthetic Merit-Order Market Simulator

This chapter documents the synthetic market simulator (`energy_storage/market/`).

3.1 Justification for Synthetic Data

A synthetic, fully observable simulator is used for four reasons: controlled experimentation, seeded reproducibility, unlimited paired evaluation episodes, and, decisive for this dissertation, regime-shift scenarios that consume no randomness, so a shifted run and its calm same-seed counterfactual are bit-identical outside the shock window. Real historical data cannot provide on-demand, repeatable regime shifts with a known counterfactual. The synthetic setting also makes the evaluation itself stronger: the hindsight optimum and the rolling-horizon oracle are exactly computable, so every policy can be scored as a fraction of the value actually attainable on the same days, a normalisation no real price history can offer. Note the trade-off explicitly: the simulator is a controlled testbed, so the claims this dissertation defends are orderings and mechanisms established within it, not absolute profit levels, and no claims transfer to real markets without validation against measured data (Section 8.2).

3.2 Exogenous Drivers

Prices form causally from a driver layer: a calendar (day-of-year, weekends, holidays), Ornstein Uhlenbeck (OU) weather processes (temperature, wind, irradiance proxies), and an OU fuel market (gas and coal prices). Shocks perturb drivers, never prices directly, so every price movement has an economic mechanism behind it.

3.3 Demand

Demand follows a heating-dominated European profile: a seasonal base load that is U-shaped over the year, with the heating peak midwinter and only modest summer cooling load, overlaid with daily shape, weekend, and holiday effects. This consumption pattern, pushed through a sloped merit order, is what produces the winter price premium. The premium *emerges* from demand meeting the supply curve rather than being painted onto prices.

3.4 The Generator Fleet

The fleet comprises solar (500 MW, the duck-curve driver), wind, must-run geothermal, a reservoir hydro plant that bids its opportunity cost as a function of reservoir fill (self-stabilising), and a deliberately granular thermal fleet: two coal units with minimum-run blocks and nine gas units spanning efficient CCGTs through old OCGT peakers. The granularity is the point: units of varying efficiency make the merit order slope, so seasonal demand moves the marginal cost. Negative prices emerge from subsidy-style negative renewable bids, coal min-run blocks, and must-run geothermal, mostly on sunny summer middays. Scarcity hours clear at the price cap and concentrate in winter and early spring, where high demand coincides with outages and low renewables.

3.5 Market Clearing

Each hour clears as a uniform-price merit-order auction: every generator submits (price, quantity) offers, the clearing intersects the resulting supply curve with demand, and all dispatched capacity is paid the marginal price. One simulated day yields 24 prices plus dispatch by technology.

3.6 Calibration

The simulator is held to a statistical calibration contract, enforced by the test suite over a seeded simulated year: median price $\approx$ \$64/MWh; a winter median more than \$6 above the summer median and a winter mean more than \$15 above it (the consumption-driven European premium; windy winter nights cap the median gap); an evening peak above overnight prices; scarcity below 1.5% of hours; and negative prices rare but present. These tests are the contract that the market has not drifted when any fleet or weather parameter changes.

3.7 Regime-Shift Scenarios

Regime shifts are expressed through five scenario types (`market/scenarios.py`): *ColdSnap* (heating-demand surge), *HeatWave* (cooling demand plus thermal derating), *FuelShock* (multiplicative gas-price step), *Drought* (reservoir inflow collapse), and *PlantOutage* (a named unit removed from the fleet). Scenarios perturb *drivers* inside a day window, never prices, and consume no randomness, so the same seed with and without a scenario differs only through the shock's causal consequences. The same scenario machinery serves both experimental roles: short windows give the unseen *transient shocks* of the robustness study (Section 6.2), and windows extended to the end of the run give the *persistent regime shifts* of the adaptation study (Section 6.3).

4 Problem Formulation: The Arbitrage MDP

This chapter formalises the battery arbitrage task as an MDP (`energy_storage/battery.py`, `energy_storage/env.py`).

4.1 Battery Physics

The battery (100 kWh, 50 kW, 90% round-trip efficiency on both charge and discharge) tracks two states: SoC, expressed as a fraction of the *current degraded* capacity $C_{\text{eff}} = C_{\text{nom}} \cdot \text{SoH}$, and SoH, which declines through two mechanisms: cycling and calendar aging:

$$\Delta \text{SoH}_{\text{cyc}} = E_{\text{thru}} \cdot \kappa_{\text{cyc}} \cdot \left(1 + s \cdot \frac{|P|}{P_{\text{max}}}\right), \quad \Delta \text{SoH}_{\text{cal}} = \kappa_{\text{cal}} \cdot (0.5 + \overline{\text{SoC}}) \cdot \Delta t,$$

where E_{thru} is the cell-side energy throughput of the step and κ_{cyc} is calibrated so that a full cycle life (5,000 equivalent full cycles) at low power takes SoH from 1.0 to the end-of-life floor of 0.8. The stress factor s makes high-power cycling cost more per kWh, standing in for the empirically established super-linear damage of aggressive cycling (Xu et al., 2018); calendar ageing ticks every hour regardless of action, calibrated to a 15-year rest life at mid SoC, and a full battery ages twice as fast as an empty one. SoH floors at 0.8 and feeds back into usable capacity; stored energy is conserved as capacity shrinks (SoC is re-expressed against the smaller capacity). Requested power is clipped to power limits and to what the SoC bounds ([0.05, 0.95]) allow, so every executed action is physically feasible by construction.

4.2 State Space

The observation is 32-dimensional: 8 scalars (SoC, SoH, sine/cosine of hour-of-day, sine/cosine of day-of-year, weekend and holiday flags) followed by the *next 24 hourly day-ahead prices*, log-normalized. Giving the agent tomorrow’s prices is not a simplification: day-ahead markets publish them, and the rolling-horizon oracle (Section 5.1) shows this window carries 99.9% of the attainable value. Weather and fuel states are deliberately *not* observed: they influence reward only through prices, which are already known exactly over the decision-relevant horizon.

4.3 Action Space

A single continuous action $a \in [-1, 1]$ maps to a power command $a \cdot P_{\text{max}}$: positive charges, negative discharges, zero idles. Infeasible commands are clipped by the physics (Section 4.1) rather than penalised, so the policy cannot damage the battery by violating operating limits; degradation, not infeasibility, is the cost channel.

4.4 Reward

The reward monetizes degradation at replacement value rather than weighting it by a free parameter:

$$r_t = (\pi_t - \Delta \text{SoH}_t \cdot c_{\text{repl}} \cdot C_{\text{nom}}) \cdot \sigma,$$

where π_t is the arbitrage cash flow of the hour (buying energy at the cleared price when charging, selling when discharging, with conversion losses applied on each side), ΔSoH_t is the health lost

this step from both ageing pathways, $c_{\text{repl}} = \$250/\text{kWh}$ is the replacement cost, and σ is a fixed reward scale. There is no tunable degradation weight and wear is priced at what it costs to buy back, so “minimizing degradation” is a single economically meaningful objective where every policy in the evaluation optimises the same dollars. Calendar ageing makes doing nothing strictly costly (an idle policy loses money), which anchors the low end of the benchmark ladder.

4.5 Episodes and Units

Episodes are 14 days (336 hourly steps) with randomised initial SoC and SoH and a burn-in of simulated days so weather and fuel states decorrelate from their initial values. The market operates in MW/MWh, the battery in kW/kWh; settlement converts between them.

5 Methodology

This chapter describes the oracles and baselines, the learned market model and its validation gate, the Dyna training procedure, and the adaptation loop (`energy_storage/oracle.py`, `market_model.py`, `market_history.py`, `adaptation.py`).

5.1 Oracles and Baselines

Every result is compared against four reference policies:

- **Hindsight optimum:** the best schedule possible, found by dynamic programming over a discretised grid of State-of-Charge values, using the actual prices for the whole episode. It is a valid upper bound because the battery is a price-taker: its trading cannot change the prices it plans against. This defines 100% of the value any policy could attain.
- **Rolling-horizon oracle:** the same dynamic program, but limited to what the agent actually knows at each step (the next 24 known prices), and re-run every hour. It reaches 99.9% of the hindsight optimum. This tells us the 24-hour window already contains almost all the information needed, so any gap left by a learned policy is a failure to learn, not a lack of information. A test pins its cost model to the real battery physics, so “optimal” really means optimal against the true dynamics. In the adaptation experiment it also serves as the *instant-adaptation baseline*: because it replans on each day’s observed prices, it adapts immediately by design.
- **Heuristic:** a simple price-threshold rule that charges when the price is in a low percentile of the known window and discharges when it is in a high one. It stands in for a trusted human operator.
- **Idle:** does nothing at all. It still loses money because the battery ages over time (calendar ageing), so it marks the floor.

Most of the arbitrage value comes from a few scarcity weeks, so profit varies a lot from one market week to the next. To keep comparisons fair, each one uses at least 20 paired episodes that share the same random seed, and results are reported as a percentage of the hindsight optimum rather than in raw dollars.

5.2 The Learned Market Model

The world model is a generative model of a single market day. It produces 24 hourly prices (log-normalized) from a 7-dimensional context: four calendar features (position in the year, weekend, holiday) and three summary statistics of the previous day’s prices (log-normalized mean, minimum, and maximum). Generation is *hour-autoregressive*, which means the model produces the day one hour at a time. The context sets the initial hidden state of a GRU (a type of recurrent neural network), which then steps through the 24 hours; each hour’s price is drawn from a Gaussian that depends on the hours already generated. To capture the model’s own uncertainty, we train an ensemble of $K = 5$ networks, each fit on a bootstrap resample of the observed days. During Dyna training, a **LearnedMarket** wrapper lets this ensemble stand in for the real market: each imagined episode picks one ensemble member and generates day after day, feeding each generated day back as the context for the next.

We also record two architectures that did not work. A flat diagonal-Gaussian day model (and a low-rank variant) fails the validation gate. Because these models treat the 24 hours as independent, they cannot represent the fact that scarcity comes in blocks of several consecutive expensive hours. To reproduce those hours at all, they have to give every hour a fat tail, which makes the daily price spread far too wide. The hour-autoregressive model avoids this: once it effectively “decides” that a day is a scarcity day, it stays on that path for the rest of the day.

5.3 The Validation Gate

Before it is used for any policy training, the fitted ensemble must pass a statistical gate. We generate synthetic years from the model and score them on the same calibration statistics the simulator itself is tested against, so the learned model has to meet the same standard as the real market. For checking price spread, the reference is a held-out real year generated from a seed that is not part of the training data.

The criteria cover the market properties that arbitrage value depends on. The synthetic median price must fall within the calibrated band. The winter price premium must show up in both the median and the mean, and evening prices must be higher than overnight prices. The mean daily spread must be within 30% of the held-out real year, which catches both models that invent too much spread and models that smooth it away. Finally, scarcity hours must stay below 1.5% of all hours, and negative-price hours must be rare but still present, so a model cannot pass while getting wrong the exact hours where most of the value is.

A model that fails any criterion is not used; the Dyna training script refuses to run unless it is given a model that has passed the gate. In standard model-based RL, trust is usually controlled by limiting how far the model is rolled out, because prediction errors compound as the policy acts. Here they cannot, because the policy cannot affect prices, so the only question is whether the generated days have the right distribution, and that can be checked *before* any RL compute is spent. The reference years come from the evaluation harness (in a real deployment this would be public price history) and are kept separate from the training budget, which only controls what the model is fit on.

5.4 Dyna Training with Planner Demonstrations

The Dyna agent is a standard SAC agent trained inside the learned market, with one addition. Before training begins, we fill part of the replay buffer with transitions from the rolling-horizon planner acting on the D stored *real* days (replayed exactly through the environment). These planner demonstrations act like the logs of a trusted existing controller: they put real-market transitions and sensible behaviour into the buffer, without the policy ever having to act on the real system before it is trained. From then on, all exploration happens in imagination. Checkpoints are chosen using only each arm’s own data source; the real simulator is used only for the final held-out evaluation.

5.5 The Adaptation Loop

This is the central mechanism (`energy_storage/adaptation.py`). The agent is deployed in a streaming setup: it sees one real market day at a time, and at the end of each day an adaptation step runs. Each night, the adaptive agent does the following:

1. **Observes and stores** the day’s 24 published prices, adding them to its history. This data arrives for free, no matter what the policy did.
2. **Detects** how surprising the day was, by computing the negative log-likelihood (NLL) of the observed day under the ensemble. A regime shift surprises every ensemble member at once, so this per-day NLL is a clearer signal than measuring how much the members disagree. The NLL over the whole deployment is itself one of the reported results. (The main version adapts every night regardless; a version that only adapts when the NLL is high is a compute-saving refinement.)
3. **Refits the market model.** It fine-tunes each ensemble member, starting from its current weights, on the most recent 90 days of observed prices plus a fixed set of pre-deployment “anchor” days that guard against forgetting. It is deliberately *not* retrained from scratch, because much of the old structure (the daily shape, calendar effects) still holds and should be kept. Each member resamples its data (bootstrap) so the ensemble stays diverse.
4. **Refreshes demonstrations.** It runs the rolling-horizon planner on the last 7 *real* (post-shift) days and adds those transitions to a small replay buffer, so the policy update is grounded in real shifted behaviour and not only in imagination.
5. **Fine-tunes the policy in imagination.** It continues SAC training from the current weights for a fixed number of steps inside the updated learned market. The imagination environment refers to the ensemble directly, so every generated day already reflects the newly refitted model.
6. **Deploys the next day.** All of the steps above run off the physical asset, overnight.

The model-free alternative, which fine-tunes SAC directly on the real price stream, is limited to just 24 new transitions per day. The gap between these two approaches is exactly what the experiment measures.

6 Experimental Setup

There are three experiments, presented in the order the argument needs them: sample efficiency under a data budget, zero-shot robustness, and adaptation to persistent shifts. The scripts are `scripts/train_dyna.py` and `scripts/run_adaptation.py`, and evaluation is shared through `energy_storage/baselines.py`.

6.1 Experiment 1: Sample Efficiency under a Real-Data Budget

Each arm is given exactly D days of real market history and nothing more:

- **online**: standard SAC trained on the real simulator and stopped after $24 \cdot D$ steps (the number of hours in D days). This shows what plain model-free learning can extract from the budget.
- **replay**: SAC trained on the same D stored days replayed exactly, but with unlimited gradient updates. This is the control that separates the effect of “more gradient updates” from the effect of “having a generative model”.
- **dyna**: SAC trained in the learned market that was fitted to the same D days (when the gate allows it), with the replay buffer seeded by planner demonstrations on those days (Section 5.4).

All arms are evaluated the same way: 20 paired 14-day episodes on the real simulator, reported as a percentage of the hindsight optimum. Varying D shows how both policy performance and gate pass/fail depend on the amount of data. An *unlimited-data* SAC reference, trained far beyond any budget, marks the model-free ceiling.

6.2 Experiment 2: Zero-Shot Robustness to Transient Shocks

The budget-trained policies are frozen and then tested under the five scenario types from Section 3.7, applied as short shock windows inside the evaluation episodes. These are regimes the policies never saw during training. Because the scenarios consume no randomness, each shocked episode can be paired with a calm episode from the same seed, which isolates the effect of the shock exactly. The question is not whether performance drops (it drops for everything) but how gracefully each policy degrades compared with the oracle on the same shocked days.

6.3 Experiment 3: Adaptation to Persistent Regime Shifts

Each deployment is one continuous 120-day market run. The first 30 days are calm and establish each arm’s baseline; then a persistent shift begins at day $T = 30$ and lasts until the end. There are three shifts, chosen to span a simple level change versus a structural change:

- **fuel-step**: the gas price doubles ($\times 2$) permanently. This is a pure change in price *level*, which the 24-hour observation window already conveys quite well, so the frozen policies should cope best here.

- **capacity-loss**: the cheap 100 MW baseload coal unit (with its minimum-run block) is permanently lost. This is a *structural* change to the merit order that alters *when* scarcity and negative prices occur, not just how high prices are.
- **cold-regime**: a lasting increase in heating demand. This shifts both the level of demand and its seasonal pattern.

The arms are: the **frozen** dyna and unlimited-data policies (never updated); **online-ft**, the unlimited-data policy fine-tuning directly on the real stream at 24 transitions per day; **adaptive-dyna**, the full nightly loop of Section 5.5; and the **heuristic**. All arms, together with the rolling-horizon **oracle**, run on the same paired seeds. Prices are exogenous, so any two arms sharing a seed see exactly the same market.

All metrics are normalized by the oracle on the same seed. Because the oracle replans on each day’s observed prices, it adapts instantly by construction, which makes it a fair “instant adaptation” ceiling:

- **capture**(t): the policy’s net profit over a trailing 7-day window divided by the oracle’s, i.e. the fraction of the instantly-adapted optimum the policy keeps;
- **recovery lag**: how many days after T it takes for capture to return to 90% of the arm’s *own* pre-shift level;
- **cumulative regret**: the total post-shift shortfall against the oracle, in dollars;
- **detection trace**: the adaptive arm’s per-day NLL, before and after each nightly fine-tune, showing whether and when the model noticed the shift.

We state our hypotheses in advance. We expect adaptive-dyna to recover within days, while online-ft takes weeks or never recovers within the 90-day horizon. We expect the frozen arms to roughly cope with the pure level shift (because the price window carries it) but never fully recover on the structural shifts. We expect the NLL trace to spike within a few days of T . And we expect the adaptive arm’s advantage to be largest on capacity-loss and smallest on fuel-step, mirroring how much of each shift the observation window already reveals.

6.4 Hyperparameters, Seeds, and Reporting

All SAC arms use the Stable-Baselines3 defaults, with one change: the random-action warmup scales with the step budget as $\min(2,000, \max(T/10, 100))$ steps, so the small online budgets are not used up entirely by warmup. The replay and dyna arms train for 500,000 environment steps across 4 parallel environments, while the online arm is capped at $24 \cdot D$ real steps by definition. Checkpoints are chosen by periodic 5-episode evaluations on each arm’s own data source, never on the real simulator. The market ensemble has $K = 5$ members, each a GRU with hidden width 128, and each is trained for 2,000 epochs on a bootstrap resample of the budget. Each night, the adaptation loop fine-tunes the ensemble on a 90-day trailing window for 25 epochs, runs 10,000 imagined SAC steps, and refreshes the planner demonstrations from the last 7 real days; the online fine-tuning arm instead takes 240 gradient steps per night on the real stream. The seed ranges never overlap: training budgets are collected at seed 0, held-out evaluation at seeds from 1,000,000, and adaptation deployments at seeds from 7,000,000. Evaluation uses at least 20 paired episodes (budget and robustness experiments) and 10 paired deployments per shift (adaptation experiment).

Dollar amounts scale with the simulator’s price cap, because most arbitrage value comes from capturing scarcity hours. We measured this sensitivity rather than assuming it: the hindsight optimum is about +\$90 per 14-day episode at a \$3000/MWh cap and +\$28 at \$1000, and at \$200 the heuristic falls below idle because spread arbitrage alone no longer covers the cost of cycling the battery. We fix the cap at \$1000/MWh for all experiments, chosen so the benchmark ladder stays strictly ordered and evaluation variance stays manageable. Every headline number is reported as a fraction of the oracle or hindsight optimum on the same seeds, which cancels out this level scaling. The learned-policy comparisons are all run at this single cap; Section 8.2 states what is and is not claimed as a result.

7 Results and Discussion

[The adaptation experiment is now complete (all three shifts, all five arms, 10 paired deployments each); the gate-table sweep, budget-sweep figure, and per-scenario robustness table remain outstanding and are marked below. Unless stated otherwise, quoted percentages are of the hindsight optimum on the real simulator, 20 paired 14-day episodes for the budget and robustness experiments, and of the same-seed rolling-horizon oracle, 10 paired 120-day deployments, for the adaptation experiment.]

7.1 Oracle Ladder and Attainable Value

The benchmark ladder on the real market frames every learned result. The hindsight optimum earns +\$28.03 per 14-day episode, which defines 100% of attainable value. The rolling-horizon oracle reaches 99.9% of this, confirming that the 24-hour observed window contains almost all the information needed. The heuristic nets −\$3.06 per episode (−10.9%): it beats idle, but it over-cycles the battery, and once its wear is priced in it ends up net-negative. Idle loses \$13.35 per episode (−47.6%) because calendar ageing cannot be avoided. The gap between the oracle and the heuristic is the value a learned policy has to play for.

7.2 The Validation Gate and the Data Threshold

Gate outcomes follow the data budget. At $D = 365$ the hour-autoregressive ensemble passes every calibration statistic and is admitted to policy training. At smaller budgets it fails, which locates a data threshold below which the learned market should not be trusted: with only a season of observed days, the model has never seen the parts of the year it would be asked to imagine. The diagonal-Gaussian architectures fail outright by making the daily spread too wide, the mechanism described in Section 5.2. The gate therefore does the job it was designed for: it turns “is the model good enough to train on?” from a judgement call into a reported table, and it answers the question before any RL compute is spent. **[PENDING: final gate table across budgets and architectures]**

7.3 Sample Efficiency under the Budget

At $D = 365$, the dyna arm reaches about 71% of hindsight, while the replay control reaches only about 39–54% on identical data and gradient budgets. So it is the generative model, not extra gradient steps, that carries the gap. The unlimited-data model-free reference reaches about

72.5%, which means one year of real data passed through the learned market recovers almost the entire model-free ceiling. [PENDING: full sweep figure across budgets]

7.4 Zero-Shot Robustness

Under the five transient shocks, the frozen dyna-365 policy keeps roughly 64–89% of the oracle’s capture on the same shocked days, depending on the shock, and its margin over the replay control roughly doubles once a shock hits. Notably, the unlimited-data baseline scores *below* dyna-365 under the fuel shock (about 56% versus about 64%). More calm-regime data does not buy robustness to shifts: both policies were shaped by the same calm regime, and adding more of it only sharpens their specialisation. This is the reason to adapt rather than simply “train longer”. [PENDING: per-scenario table and discussion of which shocks hurt most and why]

7.5 Adaptation to Persistent Shifts (Central Result)

All five arms completed all 10 paired deployments on all three shifts (150 streamed 120-day deployments in total). Table 1 reports pooled capture (ratio of summed net profit across seeds, which is stable against the near-zero single-seed oracle windows that plague a mean-of-ratios estimator), recovery lag, and cumulative post-shift regret against the instantly-adapting oracle.

Table 1: Adaptation results, all three shifts, 10 paired deployments each. Capture is pooled 7-day-windowed net profit over the same-window oracle net profit; recovery lag is days after T until an arm regains 90% of its *own* pre-shift capture (never = not within the 90-day post-shift horizon).

Shift	Arm	Pre capture	Post capture	Recovery lag	Regret (\$)
fuel-step	adaptive-dyna	66.5%	37.1%	19d	231.32
fuel-step	frozen-dyna	88.5%	53.4%	20d	172.37
fuel-step	frozen-ideal	91.8%	41.6%	never	214.43
fuel-step	online-ft	85.0%	12.3%	never	327.80
fuel-step	heuristic	54.7%	27.1%	14d	282.32
capacity-loss	adaptive-dyna	63.4%	77.7%	1d	217.66
capacity-loss	frozen-dyna	90.2%	88.7%	7d	109.96
capacity-loss	frozen-ideal	90.5%	91.1%	1d	89.55
capacity-loss	online-ft	78.2%	66.0%	9d	347.97
capacity-loss	heuristic	54.5%	55.2%	3d	466.70
cold-regime	adaptive-dyna	71.2%	74.0%	8d	856.98
cold-regime	frozen-dyna	90.3%	82.9%	1d	546.06
cold-regime	frozen-ideal	90.6%	87.6%	1d	393.53
cold-regime	online-ft	80.4%	83.8%	1d	537.72
cold-regime	heuristic	54.5%	69.8%	1d	981.52

The headline result is not the one the pre-registered hypotheses of Section 6.3 anticipated: **adaptive-dyna does not beat the frozen dyna policy on any of the three shifts, before or after the shift begins**. Figures 1–3 show why. On fuel-step, adaptive-dyna’s own pre-shift capture (66.5%) sits far below frozen-dyna’s (88.5%) — it is already behind before day $T = 30$, not just slow to recover after it. A finer, weekly breakdown of that pre-shift window separates

two effects the 30-day average blends together: capture over days 0–13 is depressed sharply (roughly 40–60% depending on shift) while the ensemble is still fine-tuning on a data-starved history, then climbs to a settled $\approx 82\%$ by days 20–29 — still 9–10 points below frozen’s flat $\approx 88\text{--}92\%$. That is, adaptive-dyna carries *two* costs: an acute startup transient in the first two weeks of any deployment, and a smaller but persistent steady-state penalty that survives settling and never closes:

Table 2: Adaptive-dyna’s settled-calm capture (days 20–29, after the startup transient) versus frozen-dyna, and both arms’ post-shift capture (days 37–119, after the shift’s own settling window).

Shift	Adaptive (settled calm)	Frozen (settled calm)	Adaptive (post-shift)	Frozen (post-shift)
fuel-step	82%	91%	37%	54%
capacity-loss	82%	92%	78%	89%
cold-regime	82%	92%	74%	83%

Against the four pre-registered hypotheses: **H1** (adaptive-dyna recovers in days, online-ft in weeks-to-never) holds only in a narrow, misleading sense — adaptive-dyna’s recovery lag is shorter than online-ft’s on fuel-step (19d vs. never) and capacity-loss (1d vs. 9d), but recovery lag is defined relative to an arm’s *own* pre-shift level, and adaptive-dyna’s own level is already depressed, so “recovering” means returning to a worse baseline, not catching frozen. **H2** (frozen arms cope with the pure level shift but not structural ones) is contradicted: frozen-dyna’s post-shift capture is *lowest* on fuel-step (53.4%, the pure level shift, hypothesized to be the easy case) and *highest* on capacity-loss (88.7%, the structural shift, hypothesized to be hard). A plausible reading is that the $2\times$ fuel-price step pushes realized prices into a level range the frozen policy’s training distribution rarely covered, while losing one generator leaves most hours’ prices close to the calm distribution even though the shift is structural; this is an interpretation, not something the experiment isolates directly. **H3** (the NLL trace spikes within days of T) is weakly supported: the detection panels of Figures 1–3 do show a reaction near the shift boundary, but the same trace spikes repeatedly during the calm pre-shift window on every shift, so raw per-day NLL is too noisy on its own to serve as a clean trigger. **H4** (adaptive’s advantage is largest on capacity-loss, smallest on fuel-step) is only partly borne out once “advantage” is read as “smallest deficit”: adaptive-dyna trails frozen-dyna by 16.3 points post-shift on fuel-step, 11.0 on capacity-loss, and 8.9 on cold-regime. The fuel-step endpoint matches the prediction — adaptive’s largest deficit (smallest advantage) is on the pure level shift — but the claim that capacity-loss would be adaptive’s *best* case does not hold: the smallest deficit is on cold-regime, with capacity-loss in between.

One robustness note carries over unchanged from the partial fuel-step result: under the fuel shock the *oracle’s own* post-shift profit rises (a bigger gas price means bigger spreads), which is why raw profit would be the wrong recovery metric and capture against the instantly-adapting oracle is the right one.

7.6 Physics-Informed Battery Dynamics

The decomposition of Section 1.4 holds the battery physics exact and learns only the market. This experiment asks the mirror-image question: if the operator had *not* calibrated their asset and had to learn the battery from a log of observed transitions, would a physics-informed surrogate (a PINN) buy sample efficiency over a plain MLP? The market is held exact throughout, so the learned battery is the only variable. Both surrogates map the environment’s own

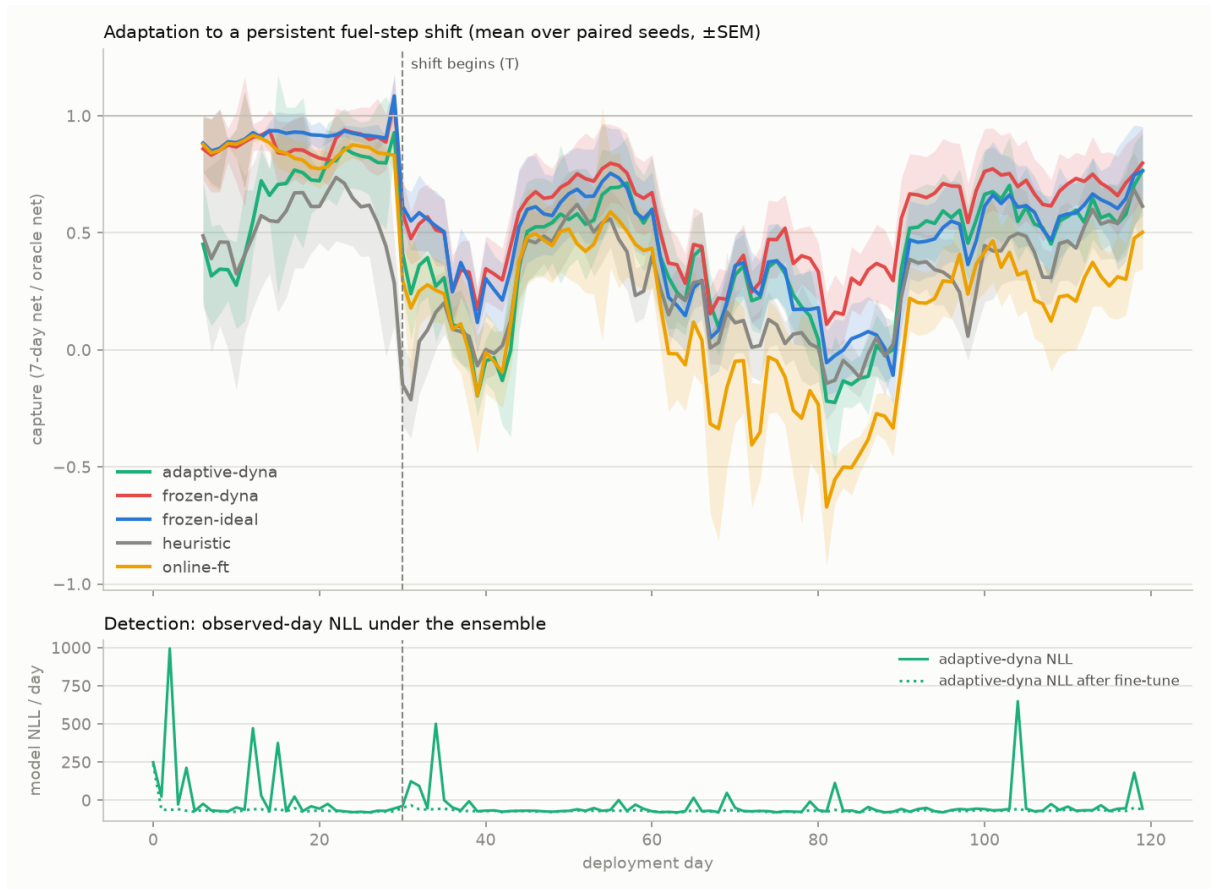


Figure 1: Capture curves and NLL detection trace, fuel-step shift, mean over 10 paired seeds ($\pm$ SEM).

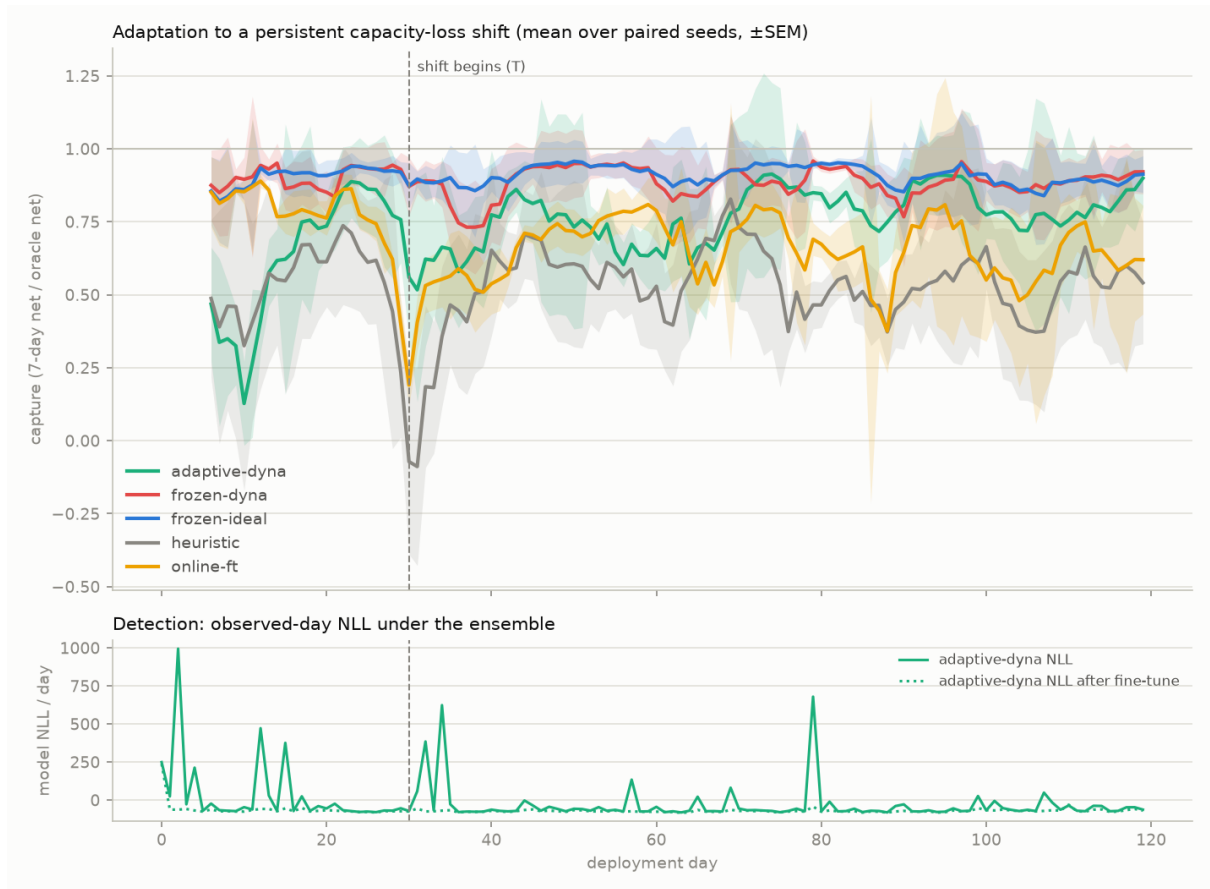


Figure 2: Capture curves and NLL detection trace, capacity-loss shift, mean over 10 paired seeds ($\pm$ SEM).

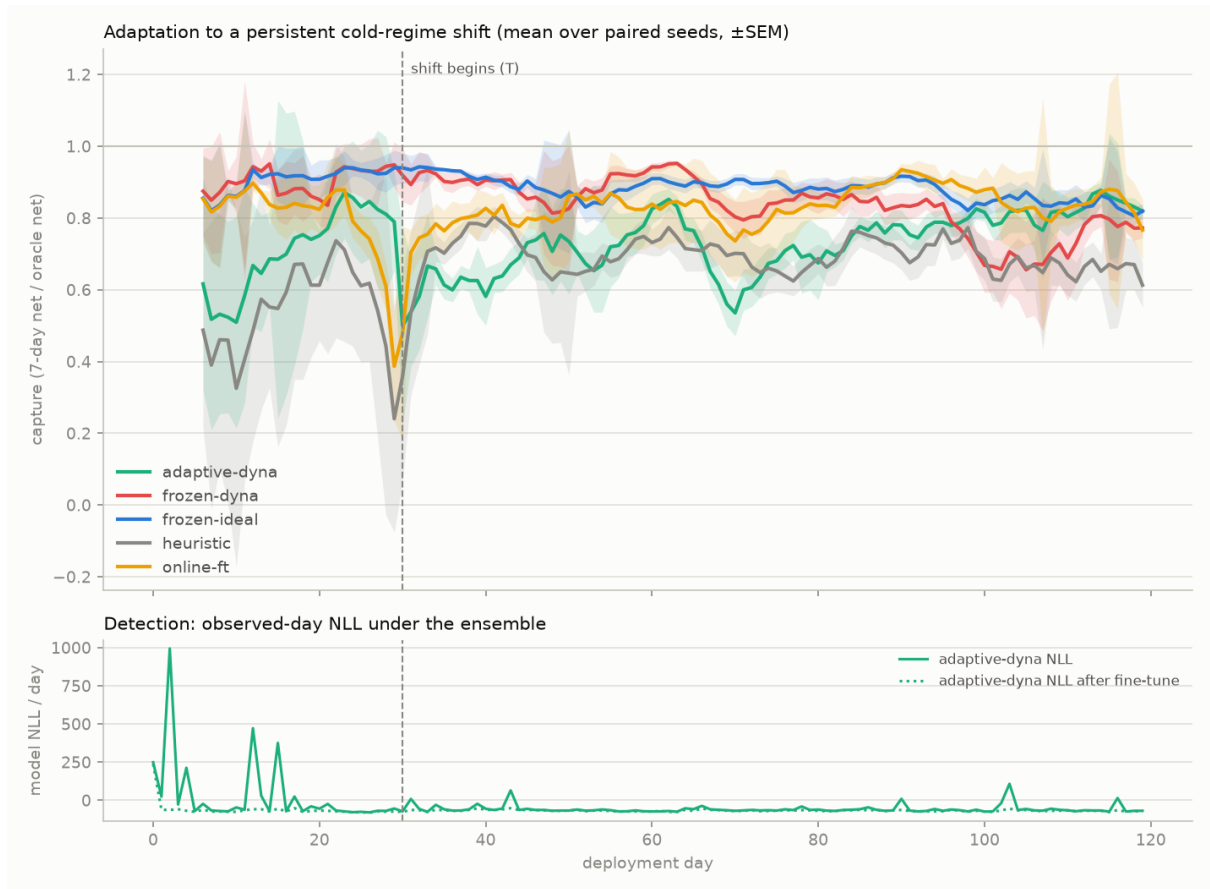


Figure 3: Capture curves and NLL detection trace, cold-regime shift, mean over 10 paired seeds ($\pm$ SEM).

step signature (soc, soh, power fraction) $\mapsto$ (Δ soc, Δ soh, grid energy) with matched trunk and parameter budget. The PINN adds physics two ways: *hard admissibility* baked into the architecture (SoH monotone non-increasing, grid-energy sign tied to the action’s sign, SoC saturated at its bounds), so it can never emit a physically impossible transition; and an optional *soft* label-free residual on collocation points (energy/efficiency conservation with known nameplate capacity, and the degradation structural equation), weighted by λ , with the unknown physical parameters — one-way efficiency and the cycle/calendar ageing rates — identified during fitting. Each arm is fitted on a shrinking budget of N transitions and scored on held-out relative error against the exact battery over the full operating envelope, plus its rate of physically-impossible predictions, averaged over three fit seeds.

The finding is negative: no robust sample-efficiency crossover, and none at all in the small- N regime a PINN is meant to own. Table 3 gives held-out pooled error under uniform coverage. A well-tuned soft residual ($\lambda = 0.3$, with a faster learning rate on the physical parameters, which are otherwise left stuck at their wrong initialisation and enforce wrong physics) edges the MLP only at the two largest budgets, where labels are plentiful enough to identify those parameters correctly and the residual acts as a mild regulariser. At every smaller budget the MLP leads: the battery map is a smooth, near-piecewise-linear function of three bounded inputs, which an unconstrained MLP interpolates from a handful of points, so there is little approximation difficulty for a prior to relieve — the regime where PINNs are known *not* to pay off. Under a realistic narrow operating band (training transitions from a cautious mid-SoC controller, tested on the full envelope) the MLP wins at every N and the soft residual, whose parameters are now identified from biased data, is uniformly worse.

Table 3: Held-out pooled relative MAE (absolute error over target std, pooled across the three targets), uniform coverage, mean over three fit seeds. **Bold** is best at each budget. **pinn-hard** is admissibility-only; **pinn-soft** adds the tuned physics residual.

N	mlp	pinn-hard	pinn-soft
500	0.018	0.019	0.014
200	0.033	0.038	0.030
100	0.044	0.061	0.066
50	0.072	0.121	0.162
25	0.142	0.147	0.198
10	0.450	0.632	0.504

The prior’s one unconditional advantage is qualitative rather than metric. The PINN’s rate of physically-impossible predictions is exactly zero at every budget and under both coverage regimes, by construction. The MLP’s climbs steeply as data thins: under uniform coverage its wrong-sign grid-energy rate reaches 2.1% at $N = 25$ and 8.6% at $N = 10$, and its free-degradation rate (a predicted *rise* in SoH) reaches 4.2% at $N = 15$; under narrow coverage the free-degradation rate reaches 15% at $N = 10$. These are exactly the exploitable errors — phantom arbitrage profit, cost-free cycling — that an optimising agent trained inside the surrogate would seek out, the same model-exploitation failure the market-side validation gate exists to prevent. So the honest reading is that a physics prior on this battery buys *admissibility, not accuracy*: a guarantee the world model never offers the policy a free lunch, independent of its average error. Whether that guarantee changes a downstream agent’s transfer — training a SAC against each surrogate and evaluating on the true battery — is the natural follow-up; the accuracy result alone does not motivate that expense, and it is left as future work.

7.7 Discussion

Two findings are already settled. The budget experiment shows that it is the generative model, not extra gradient updates, that closes the sample-efficiency gap: the replay control sees identical data and an identical gradient budget and still trails the dyna arm by 17 to 32 points. This makes the data-economics argument of Section 1.4 concrete. Once a gated generative model exists, how much training experience the agent gets is limited by compute rather than by the calendar. The robustness experiment adds that having more data is not a substitute for being able to adapt: the unlimited-data policy is *less* robust than the budget-constrained dyna policy under a fuel shock, and the partial streaming results show the same ordering under a persistent shift.

A follow-up diagnostic settles the attribution question the design anticipated: does the adaptive arm’s shortfall trace to the nightly market-model refit or to the policy fine-tune, and does nightly-always adaptation hurt through churn on calm days? The nightly ensemble refit — a trailing 90-day window plus 30 pre-stream anchor days, 25 fine-tuning epochs, continuing from current weights, exactly as configured in the adaptation loop — was replayed offline against the same fuel-step deployment (seed 7,000,000) and re-scored, after every refit, against the same calibration gate of Section 5.2 that admitted the original 365-day fit. The untouched, originally-gated ensemble’s synthetic mean daily spread is 90.2, inside the required 52.7–97.9 band ($0.7\text{--}1.3\times$ the real held-out year’s 75.29). By the third nightly refit — one week before the pre-shift period even ends, 27 nights before the shift begins — the refit ensemble’s synthetic spread is 120.4, outside the band, and it never returns inside it for the remaining 116 nights of the deployment (values from 108 to 250, full trace in Appendix A). The nightly refit is therefore never re-gated in practice: each night’s update is accepted unconditionally, and within days it drifts to a market model whose intraday price swings are systematically larger than anything the real market produces. Because the imagined SAC training happens entirely inside this ungated model, the policy is shaped nightly toward exploiting arbitrage swings that do not exist at that scale in the real market — a textbook instance of model exploitation, the same failure mode the validation gate exists to catch at the point of initial training, now recurring unchecked every night thereafter. This locates the fault on the model side: the diagnostic shows the imagined market itself becomes miscalibrated independent of what the policy does with it, though a full decomposition would additionally hold the policy fixed while only the ensemble refits, which this dissertation did not run. It also answers the churn question directly: nightly-always adaptation measurably hurts even on calm days, because the earliest refits (least real data, most data-starved) are the most miscalibrated, producing exactly the acute pre-shift transient of Table 2. Finally, it complicates the NLL-triggered variant proposed as future work (Section 8.2): since the per-day NLL trace is itself noisy pre-shift (Section 7.5), gating adaptation on ensemble calibration directly — re-running the existing validation gate after every refit and holding the previous night’s model and policy on any night it fails — is likely the more robust signal, and is cheap to add since the gate already exists and needs no new instrumentation.

8 Conclusion and Future Work

8.1 Summary of Findings

The dissertation assembles a three-timescale account of non-stationarity in storage arbitrage. The observed 24-hour price window handles *hours*: the rolling-horizon oracle attains 99.9% of hindsight, so this information is near-sufficient. A policy trained in a validated learned market

handles *days* of unseen transient shock with a graceful zero-shot floor, degrading less on the same shocked days than baselines trained on more calm data. The nightly adaptation loop is built to handle *persistent* change, at a speed set by how fast free observations accumulate rather than by on-asset interaction; evaluated in Section 7.5, it does not deliver on this promise as configured. It recovers no faster than simply freezing the pre-shift policy, and trails it throughout, because the nightly market-model refit it depends on drifts out of its own validation gate within days of streaming and is never re-checked — the central negative result of the dissertation, with a diagnosed and actionable mechanism (Section 7). Underneath all three sits the budget finding: one year of observed days, passed through a gated generative model, recovers nearly the unlimited-data model-free ceiling.

8.2 Limitations

All results are obtained in simulation, against a synthetic market and battery. The simulator is a controlled testbed, and we make no claims about transfer to a real deployment without first validating against measured price and cell data. The price-taker assumption does two jobs at once (it justifies both the decomposed world model and the hindsight oracle) and would weaken for storage large enough to move prices. The persistent-shift evaluation covers three discrete, hand-chosen regimes rather than a continuous range or gradual, compound drift. The rolling-horizon normalizer adapts instantly only because prices are fully observed a day ahead; in a market that relied on forecasts, the ceiling itself would lag. Finally, the budget-experiment policies come from a single training seed (though evaluation is multi-seed and paired).

All the quantitative results also depend on a single setting of the world: one European, heating-dominated market calibration, one price cap (\$1000/MWh), and one battery cost model (\$250/kWh replacement, 5,000-cycle life). We measure and disclose the cap dependence (Section 6.4): raw dollars scale with the cap because scarcity capture dominates arbitrage value, and the benchmark ladder stays in order from \$200 to \$3000, except that the heuristic falls below idle at the lowest cap. The learned-policy comparisons were not repeated at other caps, other market calibrations, or other degradation prices, so the *sizes* of the reported gaps should be read as specific to this setting. What the dissertation actually defends is the orderings and the mechanisms behind them: a validated generative model turns a fixed budget of observed days into unlimited training experience, and free nightly observations turn into off-asset adaptation. Grid constraints such as transmission congestion, and temperature-dependent ageing beyond the SoC-dependent calendar term, are not modelled.

8.3 Future Directions

Several directions follow directly from the adaptation loop, the most immediate motivated directly by Section 7’s diagnosis: **re-gate the nightly refit**, re-running the existing validation gate after every ensemble update and holding the previous night’s model and policy on any night it fails, and **delay adaptation** until enough streamed days have accumulated (the diagnostic locates the worst drift in the first two to three nights of any deployment) to avoid training the earliest refits on a data-starved window. Both reuse existing machinery — the gate and the streaming harness — and are cheap to implement; re-running the three-shift grid under this configuration is the natural next experiment. An NLL-*triggered* variant should also be evaluated against the always-adapt default, though Section 7 shows the raw per-day NLL trace is noisy enough pre-shift that it is a weaker candidate trigger than the calibration gate itself; the trigger is a compute optimisation, and quantifying when it is safe is open. A natural

refinement scales the nightly adaptation effort continuously with the observed surprise, so that compute concentrates where the evidence of shift is strongest. The adaptive arm also needs a transient-shock control, since adaptation should do no harm on one-day crises. Inside the nightly update, an uncertainty-weighted mixing ratio between real-day replay and imagination is unexplored. On the model side, a mixture-density day model could extend gate passes to smaller budgets, and a calibrated two-sample gate whose acceptance regions derive from the market's own year-to-year variability would replace fixed bands. The evaluation substrate suggests its own extensions: validating the simulator's shift scenarios against real market episodes such as 2022, repeating the budget sweep over multiple training seeds, and modelling other battery chemistries whose cycle and calendar ageing balance differently. Finally, relaxing the price-taker assumption toward price-making storage would couple the two halves of the world model that this dissertation deliberately kept apart.

Acknowledgements

Optional acknowledgements can be added here.

References

- Aneke, M. and Wang, M. (2016). Energy storage technologies and real life applications: A state of the art review. *Applied Energy*, 179:350–377.
- Cao, J. et al. (2020). Deep reinforcement learning-based energy storage arbitrage with accurate lithium-ion battery degradation model. *IEEE Transactions on Smart Grid*, 11(5):4513–4521.
- Chua, K. et al. (2018). Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, volume 31.
- Gama, J. et al. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37.
- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning, ICML 2018*.
- Hafner, D. et al. (2020). Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*.
- IEA (2023). Energy technology perspectives 2023. International Energy Agency.
- Janner, M. et al. (2019). When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems*, volume 32.
- Khetarpal, K. et al. (2022). Towards continual reinforcement learning: A review and perspectives. *Journal of Artificial Intelligence Research*, 75:1401–1476.
- Lecarpentier, E. and Rachelson, E. (2019). Non-stationary markov decision processes, a worst-case approach using model-based reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 32.
- Padakandla, S. (2021). A survey of reinforcement learning algorithms for dynamically varying environments. *ACM Computing Surveys*, 54(6):1–25.
- Sutton, R. S. (1991). Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 2(4):160–163.
- Xu, B. et al. (2018). Modeling of lithium-ion battery degradation for cell life assessment. *IEEE Transactions on Smart Grid*, 9(2):1131–1140.

A Additional Material

Extended results, hyperparameter tables, simulator validation plots, and supplementary analyses (e.g., the diagonal-Gaussian gate-failure diagnostics) can be placed here.

A.1 Nightly Ensemble-Refit Calibration Drift

Table 4 supports the attribution discussion of Section 7: the nightly-refit ensemble’s synthetic mean daily spread against the calibration gate’s required band ($0.7\text{--}1.3\times$ the real held-out year’s 75.29 \$/MWh, i.e., 52.7–97.9), replaying the exact refit configuration of the adaptation loop

(Section 5.5) offline against the fuel-step deployment at seed 7,000,000. Five synthetic years per checkpoint. The originally-gated ensemble (night 0) also fails the winter-mean-gap statistic under these particular resampling seeds while passing spread; since that same statistic already fails at night 0 before any fine-tuning occurs, it is read as a seed-sensitive, marginal baseline property of this ensemble rather than evidence of refit-induced drift, and spread is treated as the decisive signal.

Table 4: Synthetic mean daily spread (\$/MWh) of the nightly-refit ensemble, offline replay, fuel-step deployment (seed 7,000,000). Band: 52.7–97.9.

Night	0	1	2	3	5	10	20	30	40	60	90	119
Spread	90.2	64.9	71.0	120.4	163.3	137.3	113.2	250.5	108.1	129.3	136.3	179.8
In band?	✓	✓	✓	×	×	×	×	×	×	×	×	×